High-Performance Similarity Search Engine

Project Report

Metehan Doğan -269797012

Introduction

This project examines the performance limits of a basic Python similarity search implementation. It then evaluates ways to reduce its runtime through optimized matrix operations, dimensionality reduction, and parallel computing.

Dataset Description

The documents were generated with Llama 3.2 1B. The dataset was checked to confirm that every document was non-empty and unique. It contains 12,000 documents, with an average length of 103.47 tokens.Document and query embeddings were generated with the sentence-transformers/all-MiniLM-L6-v2 model, which produces dense vectors with 384 dimensions. Four corpus sizes were used: 1,000, 3,000, 6,000, and 12,000 documents.

The evaluation used 50 queries. Each query consists of the first 25 words from a document selected from the first 1,000 documents. The same queries were used for every corpus size to keep the performance comparisons consistent. As a result, each query's source document is included in every corpus.

Experimental environment

Most experiments ran on a local laptop with the following hardware:

- **CPU:** Intel(R) Core(TM) i5-9300H CPU @ 2.40GHz
- **RAM:** 16 GB DDR4
- **GPU:** NVIDIA GeForce GTX 1050 with 3 GB VRAM

The dataset-generation notebook used a Tesla T4 GPU in Google Colab. The local search benchmarks did not use a GPU; computations ran on the CPU.

Two project tasks were run separately in Google Colab:

- **Dataset generation and preparation:** The synthetic document dataset was generated and validated in Google Colab.
- **Vectorisation vs. loop experiments:** The vectorise_vs_loop.py experiment was run in Google Colab.

Phase 1: Baseline & Profiling

Naive Similarity Search

The initial implementation calculated cosine similarity in pure Python. The baseline used 12,000 document embeddings with 384 dimensions each and 50 queries.

The complete run took 28.394 seconds, and about 567.9 milliseconds per query.

cProfile Analysis

The cProfile results identified `cosine_similarity_python()` as the main performance bottleneck. It was called 600,000 times and accumulated 28.859 seconds of execution time.

Sorting the similarity scores for all 50 queries took about 0.267 seconds. Calls to `math.sqrt` accounted for about 0.206 seconds. Most of the runtime therefore came from calculating cosine similarity.

```
Corpus: 12000 documents
Queries: 50 queries
Dimensions: 384
============================================================
cProfile Output (sorted by cumulative time)
============================================================
         3600102 function calls in 29.812 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.588    0.588   29.812   29.812 C:\Users\dgnme\OneDrive\Masaüstü\269797012- Similarity Search Engine\phase1_baseline\naive_search.py:40(search_naive)
   600000   28.563    0.000   28.859    0.000 C:\Users\dgnme\OneDrive\Masaüstü\269797012- Similarity Search Engine\phase1_baseline\naive_search.py:20(cosine_similarity_python)
       50    0.199    0.004    0.267    0.005 {method 'sort' of 'list' objects}
  1200000    0.206    0.000    0.206    0.000 {built-in method math.sqrt}
   600050    0.098    0.000    0.098    0.000 {method 'append' of 'list' objects}
   600000    0.089    0.000    0.089    0.000 {built-in method builtins.len}
   600000    0.068    0.000    0.068    0.000 C:\Users\dgnme\OneDrive\Masaüstü\269797012- Similarity Search Engine\phase1_baseline\naive_search.py:51(<lambda>)
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}



cProfile report saved to: C:\Users\dgnme\OneDrive\Masaüstü\269797012- Similarity Search Engine\results\phase1\12k\phase1_cprofile.txt
```

Figure 1: Cprofile Output with 12000 documents.

Line Profiler Analysis

The line profiler showed that the main loop and its arithmetic operations accounted for about 99% of the time measured. The dot-product and squared-norm accumulations together accounted for roughly 25%; iterating through the loop accounted for another 24.0%.

The cosine similarity function computes the dot product and both vector norms in separate accumulations inside a Python loop.

py-spy Flame Graph Analysis

The py-spy flame graph provided a complementary, sampling-based view of the program's execution. The widest regions corresponded to search_naive() and its repeated calls to cosine_similarity_python(), with substantial sampled activity associated with the vector-processing loop. Flamegraph visually confirms that cosine similarity calculations dominate the naive search workload.

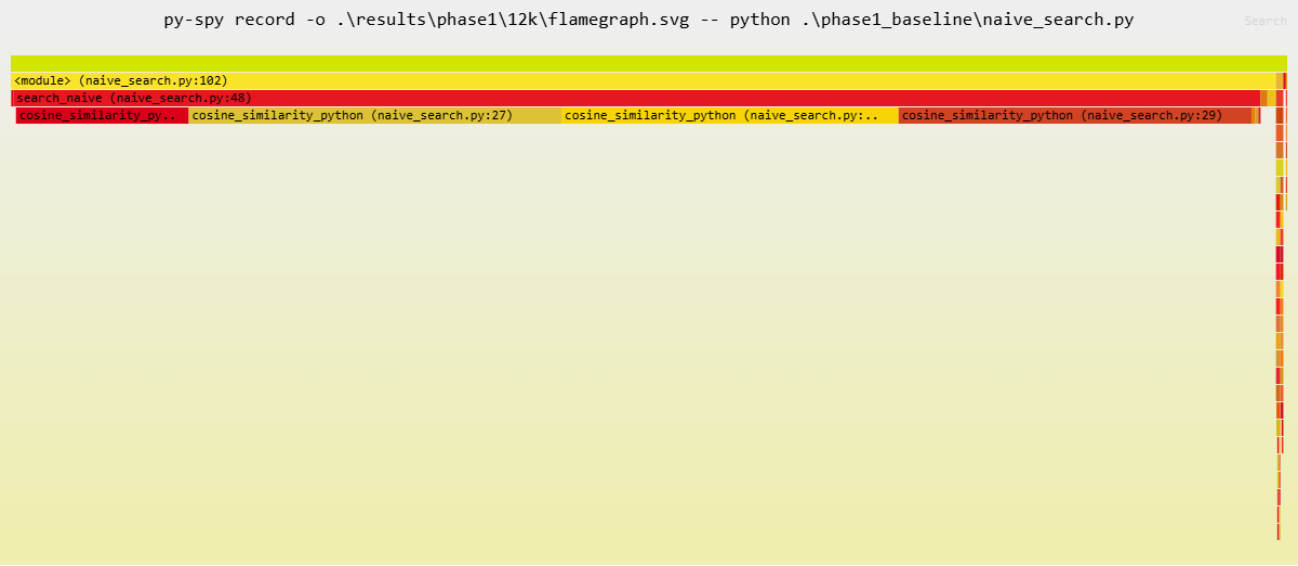


Figure 2: PySpy Flamegraph

Identified Bottlenecks

The profiling results point to three main issues:

1. **Repeated similarity calculations:** Each query is compared with all 12,000 documents, resulting in 600,000 calculations across the full workload.
2. **Python-level element-wise operations:** Each calculation loops through 384 vector dimensions to compute the dot product and both vector norms. Repeating these operations in Python adds substantial overhead.
3. **Limited sorting overhead:** Sorting takes less time than similarity computation, so it is a lower-priority target for optimization.

Phase 2: High-Performance Matrix Operations & Dimensionality Reduction

2A Sparse vs. Dense Representations

This experiment compared sparse and dense vector representations by memory use and computation time.

The 12,000-document corpus was converted to TF-IDF vectors using up to 5,000 features and English stop-word removal. The resulting sparse matrix was compared with its dense equivalent. Truncated Singular Value Decomposition (TruncatedSVD) was also applied with 128 components to create a lower-dimensional representation.

Four dot-product implementations were tested:

- Sparse–sparse multiplication using SciPy sparse operations.
- Sparse–sparse multiplication using a manual implementation.
- Sparse–dense multiplication.
- Dense–dense multiplication using NumPy.

Tests used corpus sizes of 1,000, 3,000, 6,000, and 12,000 documents. For these benchmarks, the last 100 documents were used as query texts. Memory use and computation time were recorded for each configuration.

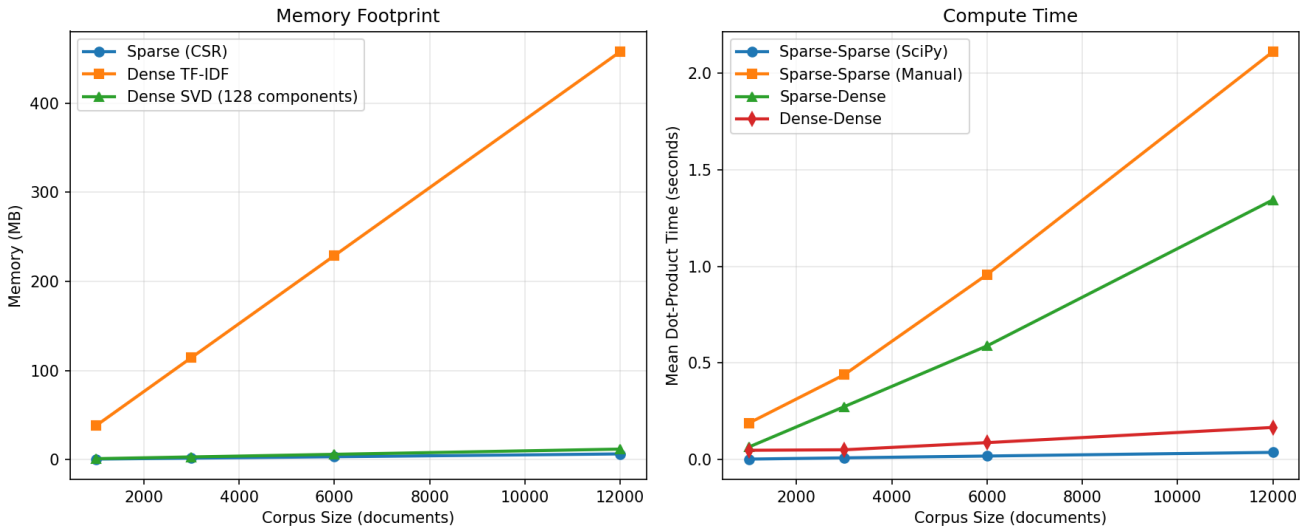


Figure 3: Memory consumption and mean dot-product computation time for different corpus sizes and multiplication methods

Table 1: Memory usage and benchmark timing statistics for corpus sizes of 1,000, 3,000, 6,000, and 12,000 documents. The table includes mean computation times and additional timing statistics recorded during benchmarking.

	Corpus Size	Sparse TF-IDF (MB)	Dense TF-IDF (MB)	Reduced SVD (MB)
0	1000	0.52	38.15	0.98
1	3000	1.57	114.44	2.93
2	6000	3.15	228.88	5.86
3	12000	6.29	457.76	11.72

	Corpus Size	Sparse-Sparse SciPy Mean (s)	Sparse-Sparse SciPy Std (s)	Sparse-Sparse Manual Mean (s)	Sparse-Sparse Manual Std (s)	Sparse-Dense Mean (s)	Sparse-Dense Std (s)	Dense-Dense Mean (s)	Dense-Dense Std (s)
0	1000	0.0032	0.0006	0.1891	0.0413	0.0655	0.0049	0.0488	0.0679
1	3000	0.0094	0.0006	0.4390	0.0188	0.2740	0.0081	0.0512	0.0210
2	6000	0.0190	0.0011	0.9574	0.0397	0.5884	0.0817	0.0882	0.0014
3	12000	0.0379	0.0009	2.1097	0.1806	1.3433	0.1131	0.1668	0.0027

Results and Discussion

Sparse and dense TF-IDF representations differed substantially in memory use. For all 12,000 documents, the sparse representation required about 6.29 MB, compared with 457.76 MB for dense TF-IDF. The dense SVD representation with 128 components required about 11.72 MB.

The multiplication methods also had different runtimes. For 12,000 documents, SciPy sparse–sparse multiplication had the lowest mean computation time, at about 0.0379 seconds. The manual sparse–sparse implementation took about 2.1097 seconds. Sparse–dense and dense–dense multiplication took about 1.3433 and 0.1668 seconds, respectively.

For this TF-IDF workload, optimized sparse operations combined low memory use with the fastest measured multiplication. Runtime still depended on the representation and implementation.

Preliminary Conclusion

For the tested TF-IDF workload, SciPy sparse operations offered the best balance of memory use and computation time. Dense representations can still be useful for operations that benefit from NumPy's matrix multiplication. The choice depends on the data and the operations being performed.

2B Dimensionality Reduction Using SVD

Objective and Methodology

This experiment compared dimensionality reduction methods by execution time and reconstruction error. Three approaches were tested:

- A custom Power Iteration SVD implementation.
- Scikit-learn TruncatedSVD.
- Scikit-learn PCA.

A reproducible random matrix with dimensions 5,000 × 768 was used for the main comparison. Each method was tested with 10, 25, 50, 100, and 200 components. Each configuration had three warm-up runs followed by five timed runs. Mean execution time and standard deviation were recorded.

Reconstruction error was calculated as the Frobenius norm of the difference between the original matrix and its rank-k approximation. Lower error means that the approximation retains more information from the original matrix.

A separate scaling experiment tested matrices of 1,000 × 768, 2,500 × 768, 5,000 × 768, and 10,000 × 768, each with 50 components.

Results and Discussion

For the 5,000 × 768 matrix, Power Iteration took about 0.99 seconds for (k=10) components and 4.96 seconds for (k=200). TruncatedSVD took about 0.20 and 0.63 seconds, while PCA took about 0.25 and 0.53 seconds, respectively.

Reconstruction error decreased as the number of components increased for all three methods. For (k=200), Power Iteration had an error of about 1,518.43, compared with 1,524.67 for TruncatedSVD and 1,537.00 for PCA.

The comparison with NumPy's rank-50 SVD approximation also produced similar errors: about 1,842.83 for Power Iteration and 1,842.78 for NumPy.

The matrix-size experiment showed the same runtime pattern. The custom Power Iteration implementation was slower than the library implementations across the tested sizes. Its execution time ranged from about 1.29 to 1.59 seconds, while TruncatedSVD and PCA generally finished sooner.

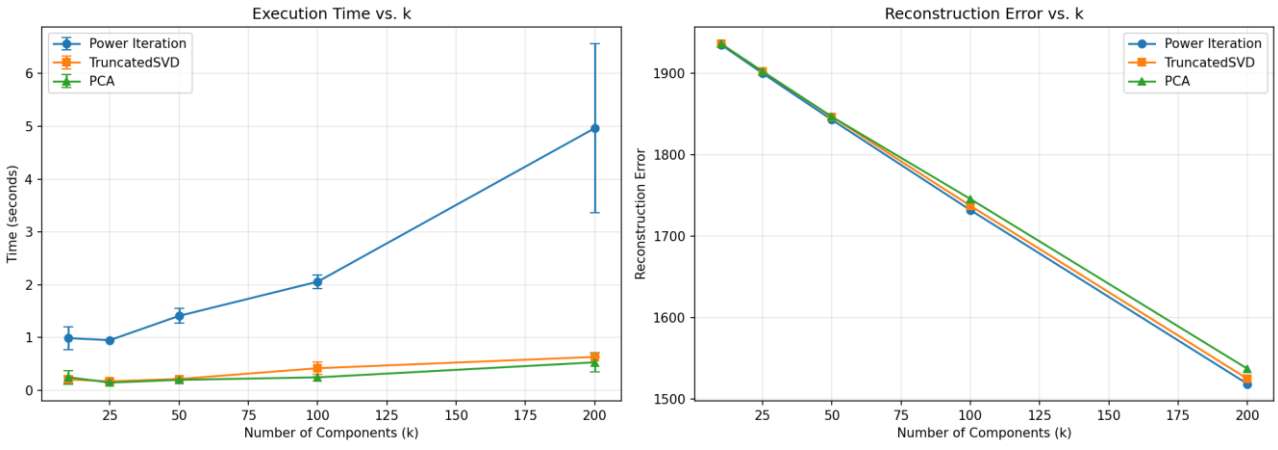


Figure 4: Execution-time comparison of the three dimensionality reduction methods for increasing matrix sizes, with the number of components fixed at 50.

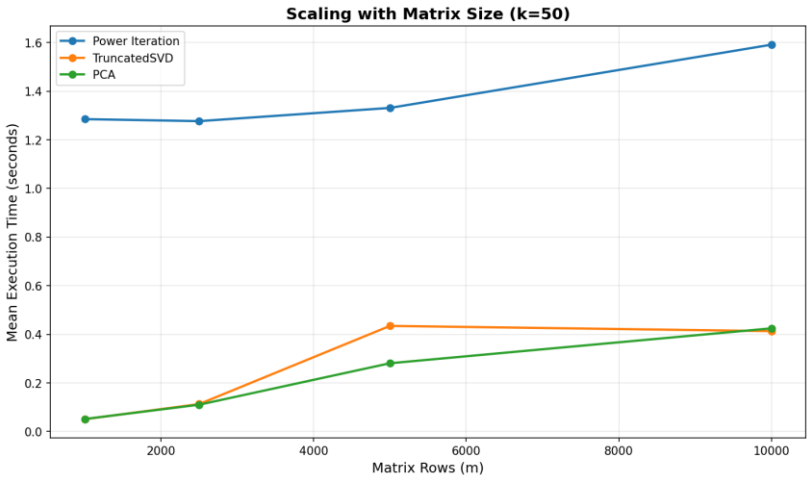


Figure 5: Execution-time comparison of the three dimensionality reduction methods for increasing matrix sizes, with the number of components fixed at 50.

Table 2: Mean execution time, standard deviation, and reconstruction error for each method and component count.

	Method	Components (k)	Mean Execution Time (s)	Standard Deviation (s)	Reconstruction Error
0	Power Iteration	10	0.9878	0.2194	1935.1130
1	Power Iteration	25	0.9497	0.0043	1900.0088
2	Power Iteration	50	1.4098	0.1394	1842.8308
3	Power Iteration	100	2.0571	0.1265	1731.9535
4	Power Iteration	200	4.9644	1.5999	1518.4279
5	TruncatedSVD	10	0.2047	0.0621	1936.3019
6	TruncatedSVD	25	0.1698	0.0348	1902.2153
7	TruncatedSVD	50	0.2141	0.0204	1846.4369
8	TruncatedSVD	100	0.4181	0.1142	1737.2520
9	TruncatedSVD	200	0.6324	0.0050	1524.6742
10	PCA	10	0.2485	0.1277	1936.2111
11	PCA	25	0.1454	0.0019	1902.2475
12	PCA	50	0.1979	0.0024	1846.6452
13	PCA	100	0.2454	0.0056	1745.5859
14	PCA	200	0.5296	0.1778	1537.0046

Preliminary Conclusion

The results show a trade-off between execution time and reconstruction error. Power Iteration had slightly lower reconstruction errors in these comparisons but took longer. TruncatedSVD ran faster and may be a better fit when computation time is the main concern.

This experiment used a random matrix rather than the actual document embeddings. It therefore compares the algorithms' computational behavior, but does not show how well they preserve semantic similarity in the document corpus. That would need to be evaluated using the actual embeddings before choosing a method for retrieval.

2C Vectorisation vs. Element-Wise vs. JIT

Four implementations of the Gram matrix operation ((G = X^T X)) were implemented and verified: element-wise Python loops, NumPy vectorisation, Numba serial, and Numba parallel using prange. All implementations passed the correctness tests. Performance was evaluated using matrices of sizes 500 × 300, 1000 × 300, 2500 × 300, and 5000 × 300.

Implementation and Experimental Results

NumPy consistently achieved the best performance across all tested matrix sizes. Compared with the pure Python implementation, NumPy achieved speedups ranging from approximately **9,000× to 12,200×**.

Numba also provided substantial improvements over pure Python by compiling loop-based operations into machine code. However, NumPy remained faster than both Numba implementations in every experiment.

An important observation was that Numba's parallel implementation did not outperform its serial implementation. For the largest matrix (5,000 × 300), Numba serial required approximately 0.9574 seconds, whereas Numba parallel required approximately 1.1350 seconds.

This result suggests that parallel execution overhead outweighed the benefits of distributing the computation across threads for the tested workload. Thread coordination, memory access patterns, and memory bandwidth can all influence the effectiveness of parallel execution.

Table 3: Comparison Table for different matrix sizes for each method.

	Matrix Size	Element-wise (s)	NumPy (s)	Numba Serial (s)	Numba Parallel (s)
0	500 × 300	14.4123	0.0016	0.0591	0.0610
1	1000 × 300	30.5480	0.0028	0.1164	0.1211
2	2500 × 300	79.6920	0.0069	0.3099	0.3705
3	5000 × 300	160.9534	0.0132	0.9574	1.1350

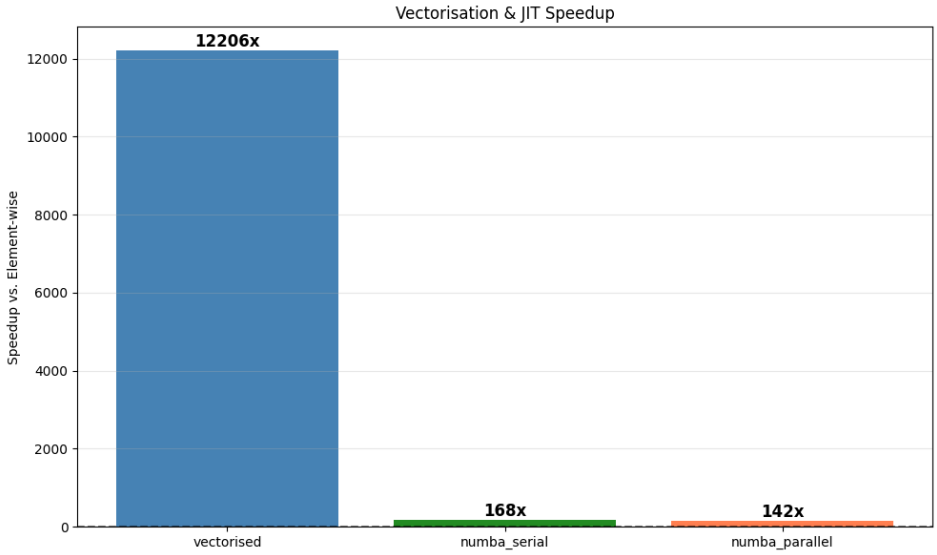


Figure 6: Speedup Comparison of NumPy and Numba Implementations o ver the Element-wise Python Baseline (5,000 × 300)

Parallelisation and Amdahl's Law

Amdahl's Law explains that parallel speedup is limited by the sequential portion of a program and the overhead associated with parallel execution. Factors such as thread coordination, memory bandwidth, and workload size can reduce the benefits of parallelisation. The results demonstrate that using multiple CPU threads does not necessarily improve performance; the effectiveness of parallel execution depends on the workload and implementation.

Conclusion

For the tested Gram matrix operation, NumPy vectorisation was the fastest approach. Numba improved the loop-based implementations, but JIT compilation and parallel execution did not outperform the optimized numerical library.

Parallel execution also did not reduce runtime in these tests. The cost of coordinating tasks and accessing memory can outweigh the benefit of using multiple threads, especially when an optimized vectorised implementation is available.

For this operation, NumPy delivered the best performance with the least implementation complexity.

Phase 3: Parallelism and concurrency

3.1 Async I/O strategy

Experimental setup

AsyncIO was evaluated in two scenarios: simulated embedding API requests and searches using pre-computed embeddings.

The API experiment used 100 queries with simulated network delays of 0.01 and 0.05 seconds. It compared serial execution, unrestricted concurrency with asyncio.gather(), and bounded concurrency using a semaphore that allowed up to 10 simultaneous tasks.

The second experiment used 50 pre-computed query embeddings and 12,000 document embeddings, each with 384 dimensions. It compared sequential NumPy, serial async execution, concurrent async execution, and AsyncIO combined with ThreadPoolExecutor.

Results: API latency

Concurrent execution reduced total runtime. With a simulated delay of 0.01 seconds, runtime fell from 1.6721 seconds to 0.1576 seconds, a 10.61× speedup. At 0.05 seconds of delay, the speedup was 27.21×.

Bounded concurrency was slower than unrestricted concurrency, with speedups of 4.89× and 7.95× for the two delays, respectively. The semaphore limited the number of simultaneous requests to ten.

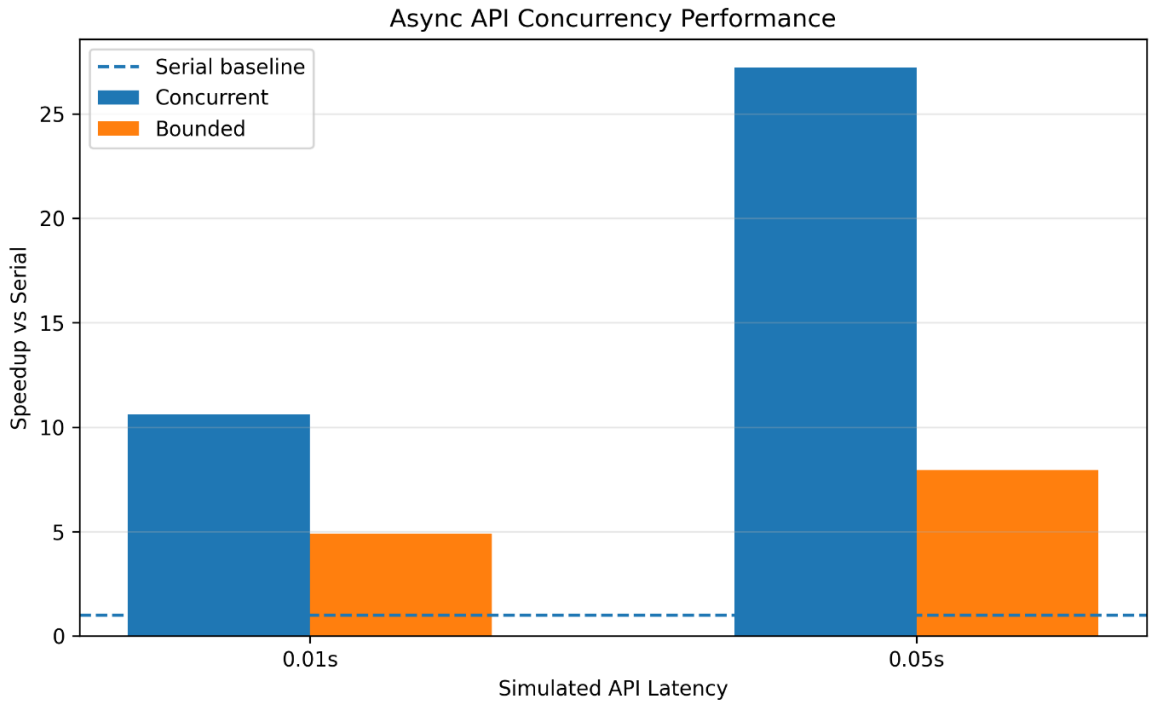


Figure 7: Async API Concurrency Performance.

AsyncIO helped when network latency was the main bottleneck. It allowed multiple requests to wait for responses at the same time instead of handling each request in sequence.

Results: pre-computed embeddings

For searches using pre-computed embeddings, sequential NumPy was fastest at 0.05256 seconds. Serial async execution took 0.05493 seconds, concurrent async execution took 0.05856 seconds, and AsyncIO combined with ThreadPoolExecutor took 0.05416 seconds.

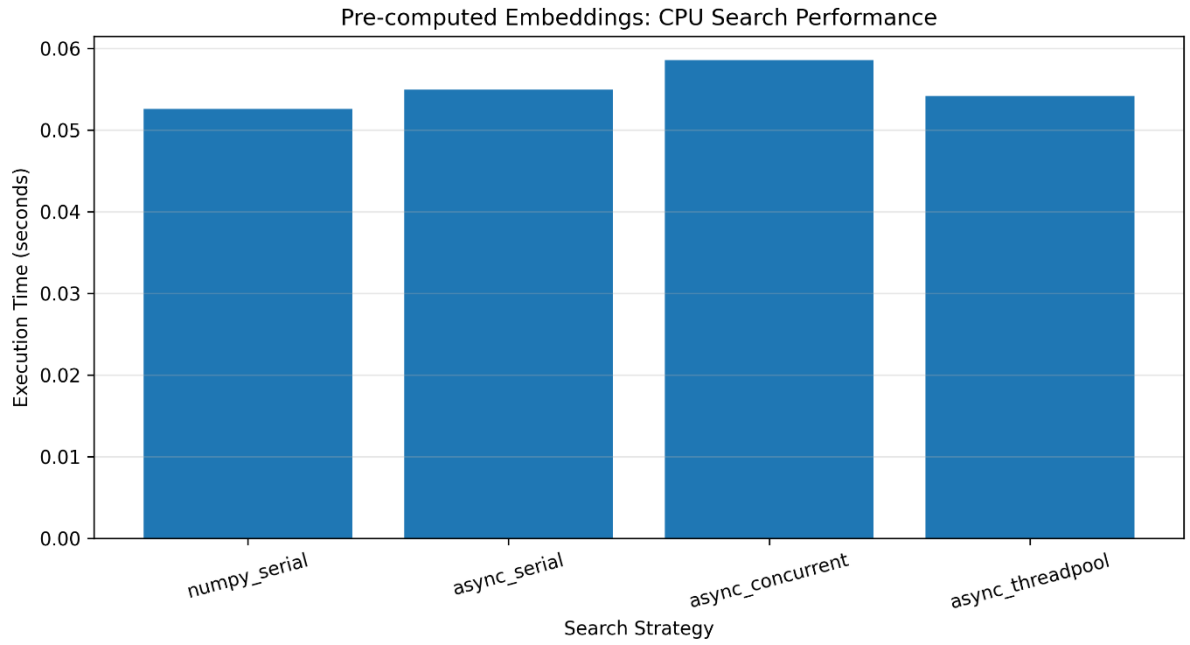


Figure 8: CPU Search performance with pre-computed embeddings.

AsyncIO did not improve this CPU-bound workload. With no network waiting, managing asynchronous tasks added overhead. Combining AsyncIO with ThreadPoolExecutor also did not outperform sequential NumPy.

Design considerations

AsyncIO is useful when an application needs to manage many concurrent I/O operations, such as embedding API requests. Bounded concurrency can help when an external service sets rate limits or resource use needs to be controlled.

For pre-computed embeddings and CPU-bound similarity searches, sequential NumPy was the most efficient approach in this experiment. AsyncIO is best suited to managing I/O; it does not replace numerical optimization or CPU parallelism.

3.2 ThreadPoolExecutor Strategy

ThreadPoolExecutor was used to run similarity search queries concurrently. The experiments used 12,000 document embeddings, 50 query embeddings, and 384-dimensional vectors. The embeddings were L2-normalized so cosine similarity could be calculated as a dot product.

The worker counts tested were 1, 2, 4, and 8. Each configuration had three warm-up iterations followed by five timed runs.

Results

Four workers gave the best performance, with an average execution time of 0.0634 seconds. This was 1.73× faster than sequential NumPy, which took 0.1099 seconds. ThreadPoolExecutor took 0.0904

seconds with one worker and 0.0692 seconds with two. With eight workers, execution time increased to 0.0698 seconds.

Compared with the single-worker configuration, four workers achieved a 1.42× speedup. With eight workers, the speedup fell to 1.29×. All configurations passed the top-5 correctness checks with a tolerance of 1e-5. The results show that performance did not scale linearly with the number of threads.

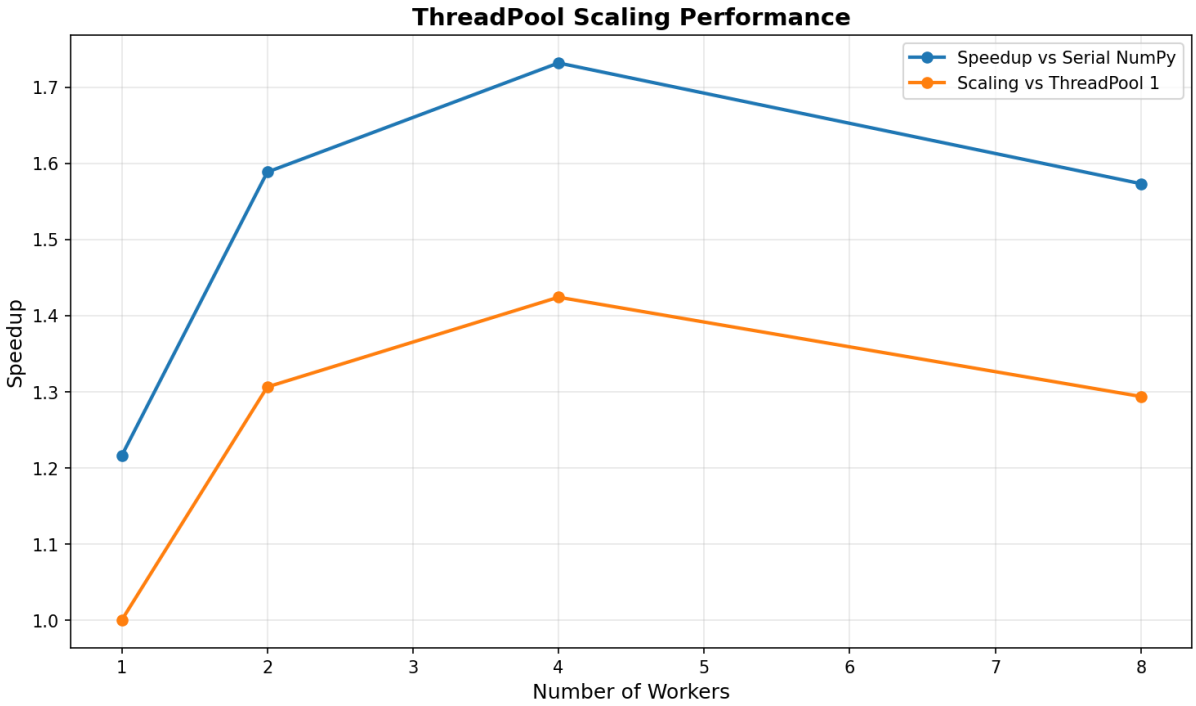


Figure 9: ThreadPool Scaling Performance.

The four-worker configuration was fastest. Increasing the count to eight raised execution time, suggesting that the added thread-management overhead outweighed any benefit from the extra workers.

GIL and BLAS considerations

The GIL limits concurrent execution of Python bytecode. NumPy's native numerical operations can release the GIL, allowing threads to perform numerical work concurrently. Thread-management overhead and competition with internally multithreaded BLAS operations can still limit scaling.

Design decision

Four ThreadPoolExecutor workers gave the best measured performance for this workload, so that configuration was selected. The results may differ with larger corpora, where memory bandwidth and thread-management overhead could affect performance differently.

3.3 ProcessPoolExecutor Strategy

ProcessPoolExecutor used shared memory to reduce the cost of transferring embedding arrays between processes. The corpus and query embeddings were copied into shared memory, and worker processes accessed them by query index range.

The experiments used 12,000 document embeddings and 50 queries, with 1, 2, 4, and 8 workers. Every configuration passed the top-5 correctness checks.

Results

Sequential NumPy was fastest, with an average execution time of 0.0549 seconds. ProcessPoolExecutor took considerably longer: 0.9650 seconds with one worker and 2.3120 seconds with eight. ThreadPoolExecutor was also slower than sequential NumPy, with execution times ranging from 0.0603 to 0.0695 seconds.

Adding workers did not improve performance. ProcessPoolExecutor's execution time increased as the worker count rose.

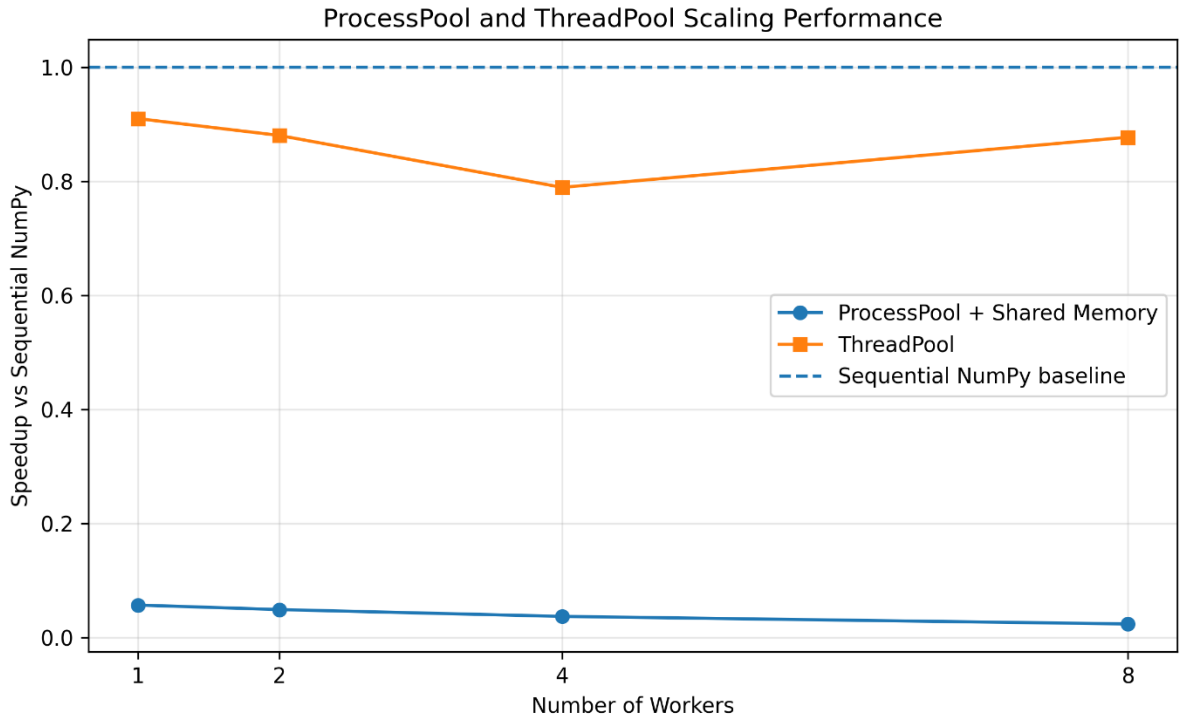


Figure 10: ProcessPool and ThreadPool Scaling Performance.

Neither threading nor multiprocessing improved on sequential NumPy for this workload. ProcessPoolExecutor remained much slower despite using shared memory. Shared memory allocation, copying the data, creating processes, and initializing workers all added overhead.

Design decision

ProcessPoolExecutor was not selected for the final similarity search implementation. Although separate processes bypass the GIL, NumPy already performs the numerical operations efficiently in native code. For this workload, process management and shared memory overhead outweighed the potential benefit of multiprocessing. Sequential NumPy was the fastest approach.

Overall design decisions

The experiments show that the value of parallelism depends on the workload.

In the initial experiment, ThreadPoolExecutor achieved a 1.73× speedup with four workers. A later comparison found that sequential NumPy could be faster than the threaded implementation. Threading therefore did not provide a consistent benefit for this workload. Results varied with task size, scheduling overhead, and how NumPy's native operations handled the work.

ProcessPoolExecutor performed poorly even with shared memory. Runtime increased with the worker count, reaching 2.312 seconds with eight workers. Process creation, shared memory initialization, and data management added more overhead than parallel execution could offset.

AsyncIO helped in the simulated API experiment, achieving speedups of 10.61× and 27.21× with API delays of 0.01 and 0.05 seconds, respectively. It did not improve CPU-bound similarity searches when the embeddings were already available.

For local similarity calculations in the current system, sequential NumPy remains the preferred approach. ThreadPoolExecutor may help with workloads where further testing shows a benefit. ProcessPoolExecutor is not justified for the current corpus size. AsyncIO is suited to managing concurrent embedding API requests and other I/O operations.

Complexity and scalability considerations

Sequential NumPy is relatively simple to implement and already provides efficient numerical operations. ThreadPoolExecutor adds task scheduling and worker management. ProcessPoolExecutor also requires process communication and shared memory handling, which increases implementation

complexity. AsyncIO uses a different programming model but is useful when the system depends on external I/O.

The design should be reassessed if the corpus grows tenfold. A larger workload may change the balance between computation and management overhead. Before choosing a parallel strategy, evaluate batch processing, memory use, BLAS configuration, and alternative indexing methods.

Choose parallelism based on the workload. In these experiments, vectorized NumPy worked best for local similarity searches, while AsyncIO was most effective for concurrent I/O.

Phase 4: Final System and Scaling

4.1 Final Search Pipeline

The final system uses precomputed 384-dimensional document embeddings and vectorized NumPy operations. It L2-normalizes the corpus and query embeddings, then calculates cosine similarity with matrix multiplication. numpy.argpartition selects the five most similar documents.

The system retains the original embedding dimensions because dimensionality reduction changed the retrieval rankings. Preserving retrieval accuracy was the priority for the final implementation.

4.2 Scaling Performance

The final implementation was benchmarked with 50 queries across four corpus sizes. The 5,000- and 10,000-document configurations were taken from the existing 12,000-embedding matrix.

Table 4: Measurement of execution time for 50 queries across four corpus sizes.

Corpus size	Mean search time (s)	Std. deviation (s)	Time per query (ms)
1,000	0.005739	0.006609	0.115
5,000	0.011894	0.006714	0.238
10,000	0.014264	0.005324	0.285
12,000	0.014610	0.004948	0.292

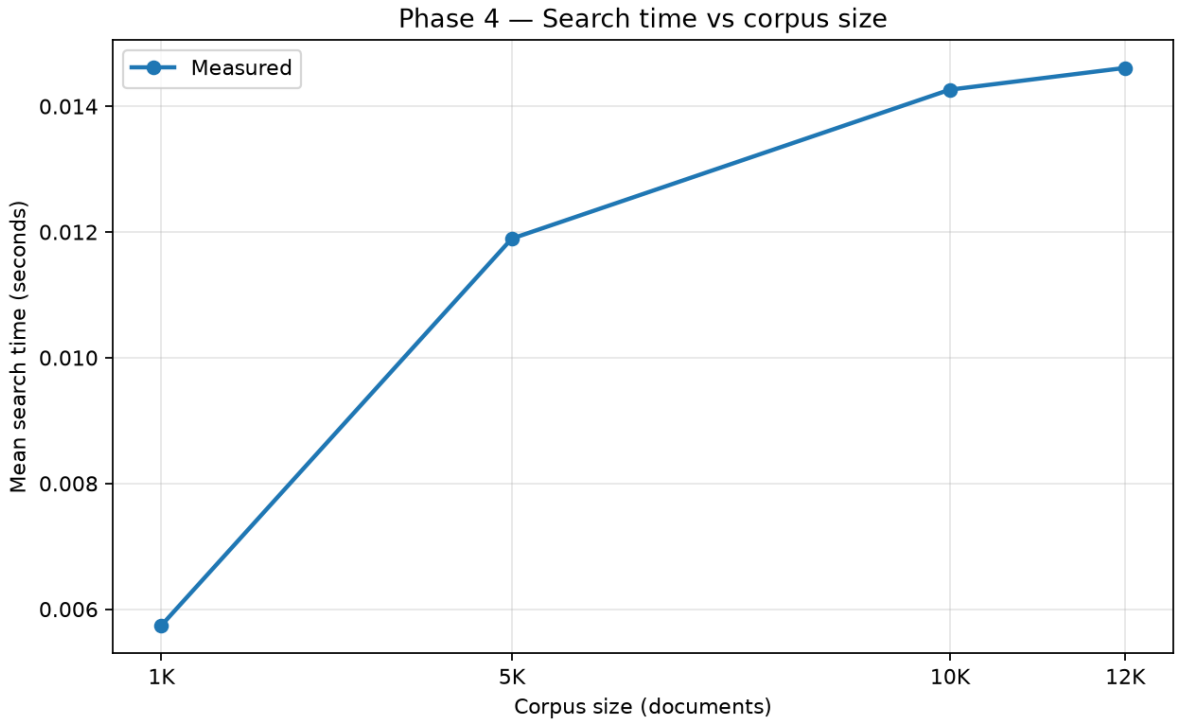


Figure 11: Search Execution Time vs. Corpus Size

Mean execution time for 50 queries increased from 0.0057 seconds with 1,000 documents to 0.0146 seconds with 12,000. The corpus grew twelvefold, while execution time increased by about 2.55 times. The standard deviations show some variation between runs, particularly for the smaller corpus configurations.

4.3 Correctness Evaluation

The optimized implementation was compared with the original Python baseline using 50 queries and top-5 retrieval.

All 250 query-level comparisons passed the correctness test The optimized implementation therefore preserved the baseline's top-5 retrieval results across all tested corpus sizes.

4.4 Baseline Performance Comparison

The Phase 1 baseline execution time was 28.394 seconds, while the final NumPy implementation completed the 12K search benchmark in 0.014610 seconds. This is an approximately 1,943× speedup over the recorded Phase 1 baseline.

4.5 Live Demonstration

The live demonstration lets users enter natural-language queries and search the 12K-document collection. The SentenceTransformer model creates an embedding for each query, while the system reuses the precomputed corpus embeddings.

For the query “How can we prevent cybersecurity threats?”, the system returned five relevant documents. All five concerned cybersecurity, with cosine similarity scores ranging from 0.6887 to 0.7033. The demonstration also displayed the retrieved documents, showing how the system handles semantic search.

Conclusion

The experiments showed that efficient similarity search requires methods suited to the workload. Parallelism did not consistently improve performance: ThreadPoolExecutor helped in some early tests, but sequential NumPy performed more reliably in later comparisons. ProcessPoolExecutor added substantial overhead.

The final implementation uses vectorized NumPy operations to search 12,000 documents efficiently. Compared with the recorded baseline, the final system was approximately 1,943 times faster and returned identical top-five results across all tested corpus sizes. Execution time also rose moderately as the corpus grew.

To extend the project, I would test larger datasets, including those with 20,000 or more documents and I would also run more controlled benchmarks to reduce timing variation.

These results apply to systems such as semantic search and retrieval-augmented generation (RAG), where relevant documents need to be retrieved quickly without sacrificing quality. The experiments show why system design needs to account for search speed, accuracy, memory use, and computational overhead.

Project Repository

The source code, experiments, and project files are available on GitHub. The repository includes implementations for each part of the project, from baseline similarity search and matrix operations to dimensionality reduction, parallel computing, and the final search system.

GitHub Repo: https://github.com/dgnmetehan/high-performance-similarity-search